HACKATHON PROJECT / 17 OCTOBER 2026




# Memory Leak Detection & Reporting Tool

Find allocated memory thatremainsunfreed—
and turn it into a clear, actionable report.

**C • glibc • LD_PRELOAD • Python**

Team: 2 Names: DSri Tanmai(2520080042) and V.Akshitha(2520080003)

TRACK • REPORT • FIX

# 1. Problem statement

- A memory leak = heap memory that is allocated but never freed
- Leaks make long-running programs slow, then crash them
- Developers need to know how much leaked and where it was allocated
- Goal: a tool that detects leaks in unmodified programs and reports them clearly

# 2. Existing tools and the gap

### Valgrind

Accurate, but 10-50x slower

### AddressSanitizer

Needs recompilation with flags

### Our tool

No recompile, low overhead, call-site report, HTML/JSON output

# 3. System design




Hash table (65,536 buckets) of live blocks: pointer, size, call stack. Mutex-protected, O(1) insert/delete.

# 4. Implementation approach

- Hooks: malloc, calloc, realloc, free, posix_memalign, aligned_alloc via dlsym(RTLD_NEXT)
- malloc: record size + backtrace(); free: remove entry
- Untracked pointer in free = double/invalid free: warn and skip (no crash)
- Thread-local busy flag stops infinite recursion; static buffer serves dlsym's calloc
- At exit a destructor dumps survivors; Python resolves file:line and groups by call site

# 5. Technologies used

**C + glibc**
dlsym, dladdr, backtrace

**pthreads**
thread-safe tracking

**Python 3**
report generator

**binutils**
addr2line

**Make + Bash**
build and tests

**Valgrind**
cross-validation

# A report should point to a useful next step

Example output format based on the supplied project results.

```
$ ./memleak-report ./tests/loop_leak

LEAKS: 100 blocks, 6400 bytes | Invalid frees: 0
#1 6400 bytes in 100 block(s) [sizes: 64]
     at leak_one() tests/loop_leak.c:4
    by main()      tests/loop_leak.c:6

Report formats: text |   HTML |   JSON
```

*Use a real screenshot from your own terminal for the final evaluation version.*

# 8. Impact, limitations, future work

## Impact

- Finds leaks without recompiling
- Points to the exact source line
- CI-friendly: --fail-on-leak

## Limitations / next

- Linux/glibc only, no mmap tracking
- Reachable-at-exit counts as leak
- Next: memory-vs-time graph, new/delete mismatch, reachability scan